

Pumas Sangrientos

Team Reference Document

Índice

1	Basics	1
1.1	Standard Competitive Programming Template	1
2	Data Structures	1
3	Geometry	1
4	Graphs	1
5	Math	1
5.1	BigInt Operations (Unsigned)	1
5.2	Binomial Coefficients Modulo Prime	2
6	Strings	2

1. Basics

1.1. Standard Competitive Programming Template

Autor: Pumas Sangrientos

Descripción: Plantilla base con optimización de I/O, macros comunes y typedefs esenciales para reducir tiempo de codificación.

```
1 #include <bits/stdc++.h>
2
3 using namespace std;
4
5 // === Typedefs ===
6 typedef long long ll;
7 typedef long double ld;
8 typedef pair<int, int> pii;
9 typedef pair<ll, ll> pll;
10 typedef vector<int> vi;
11 typedef vector<ll> vll;
12
13 // === Macros ===
14 #define endl '\n'
15 #define pb push_back
16 #define all(x) (x).begin(), (x).end()
17 #define sz(x) (int)(x).size()
18 #define FOR(i, a, b) for(int i = (a); i < (b); ++i)
19 #define ROF(i, a, b) for(int i = (a); i >= (b); --i)
20
21 void fast_io() {
22     ios_base::sync_with_stdio(false);
23     cin.tie(NULL);
24     cout.tie(NULL);
25 }
```

```
26 }
27
28 void solve() {
29     // Escribe tu lógica aquí
30 }
31
32 int main() {
33     fast_io();
34
35     int t = 1;
36     // cin >> t; // Descomentar si el problema tiene múltiples casos de prueba
37     while(t--) {
38         solve();
39     }
40
41     return 0;
42 }
```

2. Data Structures

3. Geometry

4. Graphs

5. Math

5.1. BigInt Operations (Unsigned)

Autor: botellot / ICPC BRO

Descripción: Basic BigInt structure for positive integers using base 10^9 .

```
1 struct BigInt {
2     static const int BASE = 1e9;
3     vector<int> nodes;
4     BigInt() {}
5     BigInt(long long v) {
6         if (v == 0) nodes.push_back(0);
7         while (v > 0) {
8             nodes.push_back(v % BASE);
9             v /= BASE;
10        }
11    }
12    BigInt(string s) {
13        for (int i = (int)s.length(); i > 0; i -= 9) {
14            if (i < 9)
15                nodes.push_back(stoi(s.substr(0, i)));
16            else
17                nodes.push_back(stoi(s.substr(i - 9, 9)));
18        }
19        trim();
20    }
21    void trim() {
22        while (nodes.size() > 1 && nodes.back() == 0)
23            nodes.pop_back();
24        if (nodes.empty()) nodes.push_back(0);
25    }
26 };
```

```

27 // --- OPERACIONES EXTERNAS ---
28 // O(N) - Suma estándar con carry
29 BigInt sum_lnum(const BigInt& a, const BigInt& b) {
30     BigInt res = a;
31     int carry = 0;
32     for (size_t i = 0; i < max(res.nodes.size(), b.nodes.size()) || carry; ++i)
33     {
34         if (i == res.nodes.size()) res.nodes.push_back(0);
35         long long cur = (long long)res.nodes[i] + carry + (i < b.nodes.size() ?
36             b.nodes[i] : 0);
37         res.nodes[i] = cur % BigInt::BASE;
38         carry = cur / BigInt::BASE;
39     }
40     return res;
41 }
42 // O(N) - Resta estándar (Asume a >= b)
43 BigInt less_lnum(const BigInt& a, const BigInt& b) {
44     BigInt res = a;
45     int carry = 0;
46     for (size_t i = 0; i < b.nodes.size() || carry; ++i) {
47         long long cur = res.nodes[i] - carry - (i < b.nodes.size() ? b.nodes[i]
48             : 0);
49         carry = cur < 0;
50         if (carry) cur += BigInt::BASE;
51         res.nodes[i] = (int)cur;
52     }
53     res.trim();
54     return res;
55 }
56 // O(N) - Multiplicación por escalar
57 BigInt prod_lnum_int(const BigInt& a, ll v) {
58     if (v == 0) return BigInt(0);
59     BigInt res = a;
60     ll carry = 0;
61     for (size_t i = 0; i < res.nodes.size() || carry; ++i) {
62         if (i == res.nodes.size()) res.nodes.push_back(0);
63         long long cur = res.nodes[i] * 1LL * v + carry;
64         res.nodes[i] = cur % BigInt::BASE;
65         carry = cur / BigInt::BASE;
66     }
67     res.trim();
68     return res;
69 }
70 // O(N*M) - Multiplicación BigInt * BigInt
71 BigInt prod_lnum(const BigInt& a, const BigInt& b) {
72     BigInt res;
73     res.nodes.resize(a.nodes.size() + b.nodes.size(), 0);
74     for (size_t i = 0; i < a.nodes.size(); ++i) {
75         ll carry = 0;
76         for (size_t j = 0; j < b.nodes.size() || carry; ++j) {
77             long long cur = res.nodes[i + j] +
78                 a.nodes[i] * 1LL * (j < b.nodes.size() ? b.nodes[j]
79                     : 0) + carry;
80             res.nodes[i + j] = cur % BigInt::BASE;
81             carry = cur / BigInt::BASE;
82         }
83     }
84     res.trim();
85     return res;
86 }
87 // O(N) - División por escalar
88 BigInt div_lnum_int(const BigInt& a, ll v) {
89     BigInt res = a;
90     ll carry = 0;
91     for (int i = (int)res.nodes.size() - 1; i >= 0; --i) {

```

```

88         long long cur = res.nodes[i] + carry * 1LL * BigInt::BASE;
89         res.nodes[i] = (int)(cur / v);
90         carry = cur % v;
91     }
92     res.trim();
93     return res;
94 }
95 // O(N) - Módulo por escalar
96 ll mod_lnum_int(const BigInt& a, ll v) {
97     ll m = 0;
98     for (int i = (int)a.nodes.size() - 1; i >= 0; --i)
99         m = (a.nodes[i] + m * 1LL * BigInt::BASE) % v;
100     return m;
101 }
102 // O(N) - Print formateado con ceros a la izquierda para la base
103 void print(const BigInt& a) {
104     if (a.nodes.empty()) { printf("0\n"); return; }
105     printf("%d", a.nodes.back());
106     for (int i = (int)a.nodes.size() - 2; i >= 0; --i)
107         printf("%09d", a.nodes[i]);
108     printf("\n");
109 }

```

5.2. Binomial Coefficients Modulo Prime

Autor: botellot

Descripción: Calcula coeficientes binomiales (nCr) usando factoriales precomputados e inversos modulares. Requiere que m sea primo.

```

1 const int MAXN = 1000005; // Ajustar según el problema
2 // 0 998244353
3 ll factorial[MAXN];
4 ll inv_factorial[MAXN];
5 // Exponenciación binaria para el inverso modular O(log m)
6 ll power(ll base, ll exp) {
7     ll res = 1;
8     base %= m;
9     while (exp > 0) {
10         if (exp % 2 == 1) res = (res * base) % m;
11         base = (base * base) % m;
12         exp /= 2;
13     }
14     return res;
15 }
16 ll modInverse(ll n) {
17     return power(n, m - 2);
18 }
19 // Precomputo en O(MAXN)
20 void precompute() {
21     factorial[0] = 1;
22     for (int i = 1; i < MAXN; i++)
23         factorial[i] = (factorial[i - 1] * i) % m;
24     // Inverso del factorial final usando Fermat
25     inv_factorial[MAXN - 1] = modInverse(factorial[MAXN - 1]);
26     // Inversos de los demás factoriales en O(MAXN)
27     for (int i = MAXN - 2; i >= 0; i--)
28         inv_factorial[i] = (inv_factorial[i + 1] * (i + 1)) % m;
29 }
30 // Cálculo en O(1)
31 ll nCr(int n, int k) {
32     if (k < 0 || k > n) return 0;
33     return factorial[n] * inv_factorial[k] % m * inv_factorial[n - k] % m;
34 }

```

6. Strings